

Consignes générales:

- À remettre un dossier compressé de votre projet complet avec la base de données utilisée et ce document contenant les réponses aux exercices.
- Date de remise : vous avez jusqu'au début du deuxième cours de la semaine 15 pour le remettre

Partie 1

Insérez le code de chaque exercice et une capture de l'exécution du script sur votre machine.

Exercice 1 : Validation et boucle (5 pts)

Écrire un script qui :

1. demande à l'utilisateur de saisir un entier positif
2. tant que la valeur saisie est invalide (non entier ou négatif), redemande la saisie
3. une fois valide, affiche la somme des entiers de 1 jusqu'à ce nombre

```
GNU nano 8.1                                                                 exercicel.py
while True:
    val = input("Entrez un entier positif : ")

    try:
        n = int(val)

        if n <= 0:
            print("Erreur : le nombre doit être positif.")
        else:
            total = 0

            for i in range(1, n + 1):
                total = total + i

            print("La somme des entiers de 1 jusqu'à", n, "est :", total)
            break

    except ValueError:
        print("Erreur : vous devez entrer un entier valide.")

romeoserveur@localhost:~/Partiel1_Travail2$ python3 exercicel.py
Entrez un entier positif : adk
Erreur : vous devez entrer un entier valide.
Entrez un entier positif : -5
Erreur : le nombre doit être positif.
Entrez un entier positif : 0
Erreur : le nombre doit être positif.
Entrez un entier positif : 5
La somme des entiers de 1 jusqu'à 5 est : 15
```

Exercice 2 : Fonctions et structure (5 pts)

Créer un programme structuré contenant notamment :

- une fonction `est_pair(n)` qui retourne au programme principal `True` ou `False`
- une fonction `afficher_pairs(liste)` qui affiche uniquement les nombres pairs

Dans le programme principal :

- Menu pour appeler les deux fonctions et une option Quitter
- Avant d'appeler la deuxième fonction, demander 5 nombres à l'utilisateur, les stocker dans une liste qui sera utilisée dans l'appel.

```
def pair(n):
    return n % 2 == 0

def afficherpair(nums):
    print("Nombres pairs :")
    for n in nums:
        if pair(n):
            print(n)

def lireentier(msg):
    while True:
        try:
            n = int(input(msg))
            return n
        except ValueError:
            print("Erreur : entrez un entier valide.")

choix = ""
while choix != "3":
    print("\nMenu")
    print("1. Vérifier si un nombre est pair")
    print("2. Afficher les nombres pairs d'une liste")
    print("3. Quitter")
    choix = input("Entrez votre choix : ")

    if choix == "1":
        n = lireentier("Entrez un nombre : ")
        if pair(n):
            print(n, "est pair.")
        else:
            print(n, "est impair.")
    elif choix == "2":
        nums = []
        for i in range(5):
            n = lireentier("Entrez un nombre : ")
            nums.append(n)
        afficherpair(nums)
    elif choix == "3":
        print("Fin du programme.")
    else:
        print("Choix invalide.")
```

```
romeoserveur@localhost:~/Partie1_Travail2$ python3 exercice2.py
```

Menu

1. Vérifier si un nombre est pair
2. Afficher les nombres pairs d'une liste
3. Quitter

Entrez votre choix : 1

Entrez un nombre : 8

8 est pair.

Menu

1. Vérifier si un nombre est pair
2. Afficher les nombres pairs d'une liste
3. Quitter

Entrez votre choix : 2

Entrez un nombre : 1

Entrez un nombre : 2

Entrez un nombre : 3

Entrez un nombre : 4

Entrez un nombre : 5

Nombres pairs :

2

4

Menu

1. Vérifier si un nombre est pair
2. Afficher les nombres pairs d'une liste
3. Quitter

Entrez votre choix : 3

Fin du programme.

◆ Exercice 3 : Manipulation de fichiers (5 pts)

Écrire un script qui :

1. demande à l'utilisateur un nom de fichier
2. vérifie si le fichier existe
 - si oui :
 - a. affiche le nombre de lignes
 - b. affiche le nombre de mots
 - sinon :
 - c. affiche un message d'erreur et demande à l'utilisateur de saisir un chemin de fichier valide. Le chemin doit un fichier et non un dossier.

```
romeoserveur@localhost:~/Partie1_Travail2 – sudo vi exercice3.py
~/Partie1_Travail2

import os

while True:
    path = input("Entrez le chemin du fichier : ")

    if not os.path.exists(path):
        print("Erreur : le chemin n'existe pas.")
        continue

    if not os.path.isfile(path):
        print("Erreur : le chemin doit être un fichier, pas un dossier.")
        continue

    f = open(path, "r", encoding="utf-8")
    lignes = f.readlines()
    f.close()

    nb_lignes = len(lignes)
    nb_mots = 0

    for ligne in lignes:
        mots = ligne.split()
        nb_mots = nb_mots + len(mots)

    print("Nombre de lignes :", nb_lignes)
    print("Nombre de mots :", nb_mots)
    break

romeoserveur@localhost:~/Partie1_Travail2$ python3 exercice3.py
Entrez le chemin du fichier : dsfiusfs.txt
Erreur : le chemin n'existe pas.
Entrez le chemin du fichier : .t
Erreur : le chemin n'existe pas.
Entrez le chemin du fichier : test.txt
Nombre de lignes : 3
Nombre de mots : 12
```

Partie 2

Conception de l'application

Concevoir une application python qui va utiliser votre base de données AssuranceVotrenom créée durant le TP1.

```
MariaDB [(none)]> USE assurance_nom;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A
```

Database changed

```
MariaDB [assurance_nom]> SHOW TABLES;  
+-----+  
| Tables_in_assurance_nom |  
+-----+  
| Employe                  |  
| Entreprise               |  
| TelephoneEntreprise      |  
+-----+  
3 rows in set (0,000 sec)
```

Ajoutez, s'ils n'y sont pas, les champs codePostal (et dateRecrutement à la table employé.

```
MariaDB [assurance_nom]> ALTER TABLE Employe  
-> ADD COLUMN codePostal VARCHAR(7),  
-> ADD COLUMN dateRecrutement DATE;  
Query OK, 0 rows affected (0,012 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
MariaDB [assurance_nom]> ALTER TABLE Entreprise  
-> ADD COLUMN AdresseEntreprise VARCHAR(150);  
Query OK, 0 rows affected (0,005 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

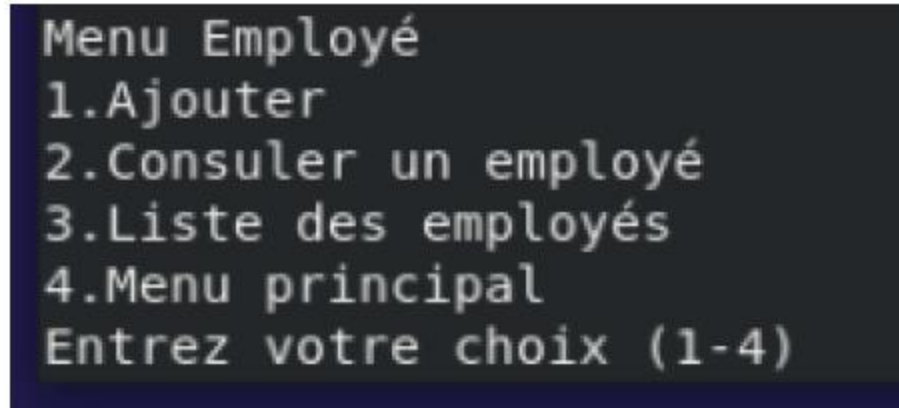
- a. À l'exécution de l'application, l'utilisateur est invité à rentrer son nom et son mot de passe. Après vérification dans le fichier /etc/passwd, la possibilité est donnée à l'utilisateur de poursuivre si l'utilisateur y est présent. Autrement, l'utilisateur est invité à entrer un nom d'utilisateur et un mot de passe valides (Pas besoin de tester le mot de passe mais sa saisie doit être cachée). Au bout de trois tentatives infructueuses, l'application s'arrête. (5 pts)
- b. Les utilisateurs ont accès à un menu principal qui propose six options :

```
Application Assurance
1.Entreprise
2.Employé
3.Sauvegarde
4.Restauration
5.Documentation
6.Quitter
Entrez votre choix (1-6)
```

1. Entreprise qui doit permettre l'ajout d'une entreprise, la modification de l'adresse d'une entreprise via son Id, la suppression d'une entreprise via son Id, la liste des entreprises et la liste des employés d'une entreprise dont l'identifiant est saisi au moment de l'exécution. (20 pts)

```
Menu Entreprise
1.Ajouter
2.Modifier l'adresse
3.Supprimer
4.Liste des entreprises
5.Liste des employés d'une entreprise
6.Menu principal
Entrez votre choix (1-6)
```

2. Employé qui doit permettre l'ajout d'un employé, la consultation des informations d'un employé via son id et la liste des employés. (20 pts)



3. Sauvegarde qui doit permettre de sauvegarder la base de données (5 pts)
4. Restauration qui doit permettre de restaurer la base de données. (4 pts)
5. Documentation (10 pts)
6. Quitter qui est la seule option qui doit permettre d'arrêter l'application. (1 pt)

Spécifications :

- Lorsqu'un utilisateur choisit l'option 1 du menu principal, un sous-menu Entreprise est affiché avec les options d'ajout, modification d'adresse, de suppression d'une entreprise, liste des employés d'une entreprise ainsi qu'une option permettant de retourner au menu principal
- Lorsqu'un utilisateur choisit l'option 2 du menu principal, un sous-menu employé est affiché avec les options d'ajout, consultation d'un employé et liste des employés ainsi qu'une option de retour au menu principal
- Lorsqu'un utilisateur choisit l'option 3 du menu principal, la base de données est sauvegardée et après un message indiquant que la sauvegarde a été correctement faite, le menu principal est réaffiché. L'utilisateur qui exécute l'application doit pouvoir visualiser le message (temporiser avant de retourner au menu principal)
- Lorsqu'un utilisateur choisit l'option 4 du menu principal, la base de données est supprimée, recrée (son nom) puis restaurée à partir d'une sauvegarde existante. On vérifie d'abord qu'un fichier sql au nom de la base de données existe avant de supprimer la base.
- Lorsqu'un utilisateur choisit l'option 5 du menu principal, un menu Documentation des fonctionnalités est affiché. Lorsque l'utilisateur sélectionne une option, la documentation relative à la fonctionnalité est affichée à l'écran.

- Lorsqu'un utilisateur choisit l'option 6 du menu principal, un message indiquant la sortie de l'application et l'application s'arrête.
- Vous devez faire la validation de date, du numéro de téléphone et du code postal.
- Vous devez gérer les exceptions pour éviter que l'application s'arrête en cas d'erreur de l'utilisateur.
- Ajoutez un commentaire avant le début de chaque section de votre code.
- L'affichage des résultats des différentes options des menus doit être bien agencé (ergonomique). Aussi, s'assurer d'effacer l'écran avant d'afficher de nouveaux éléments (Menus, messages, résultats...).
- On assume que l'utilisateur qui exécute l'application est un utilisateur ajouté au préalable sur le serveur MariaDb avec les autorisations nécessaires sur la base de données.

Validation :

Durant la deuxième séance de la semaine 15, une validation individuelle sera faite en classe sur votre machine. (20 points)